Ejercicio 1

Quiero implementar en Java 21 una funcion recursiva para calcular el factorial de un numero entero. Este problema se puede resolver de manera recursiva ya que el factorial de n se define como n multiplicado por el factorial de n-1. El caso base ocurre cuando n es igual a 0 o 1, devolviendo directamente 1 porque por definicion matematica el factorial de esos valores es 1 y evita que la funcion descienda indefinidamente. En el caso recursivo el problema se reduce decrementando n en 1 en cada llamada (n * factorial(n - 1)). La funcion debe devolver el resultado numerico final calculado. El codigo debe incluir comentarios claros explicando el caso base, el caso recursivo y la pila de llamadas (call stack). El codigo debe tener manejo de excepciones y poder manejar casos donde el numero ingresado sea negativo. Tambien quiero probarlo con los siguientes casos de prueba:

- factorial(0)
- factorial(1)
- factorial(5)
- factorial(-3) para ver el manejo de error o excepcion

Ejercicio 2

Quiero implementar en Java 21 una funcion recursiva que sume todos los numeros enteros desde n hasta 1. El problema se resuelve recursivamente porque la suma de n terminos se puede plantear como la suma del numero actual mas la suma acumulada de los n-1 numeros anteriores (n + suma(n - 1)). El caso mas simple y caso base es cuando n vale 1 devolviendo 1, pero si la entrada n es 0 la funcion debe devolver directamente 0 para evitar errores. En el caso recursivo el problema se reduce restando 1 al parametro en cada paso acercandose a la condicion de corte. La funcion debe retornar el entero con la sumatoria final calculada. El codigo debe incluir comentarios sobre el funcionamiento de la recursion y la pila de llamadas. Quiero validar que la funcion funciona correctamente con estos casos:

- suma(0)
- suma(1)
- suma(5)
- suma(10)

Ejercicio 3

Quiero implementar en Java 21 una funcion recursiva para multiplicar dos numeros a y b sin utilizar el operador de multiplicacion *. La multiplicacion se puede representar recursivamente como sumas repetidas de a sumado b veces. El parametro que se reduce en cada invocacion es b, restandole 1 por iteracion mientras sumamos a (a + multiplicar(a, b - 1)). El caso base debe ser cuando el factor b vale 0 devolviendo 0, ya que cualquier numero multiplicado por cero da cero y frena la llamada. La funcion debe devolver el producto acumulado total. El codigo debe tener comentarios explicando el caso base, el caso recursivo y la evolucion de la pila de memoria. Quiero probarlo con estos casos:

- multiplicar(6, 0)
- multiplicar(5, 1)

- multiplicar(4, 3)
- multiplicar(7, 4)

Ejercicio 4

Quiero implementar en Java 21 una funcion recursiva que calcule la potencia de una base elevada a un exponente sin usar Math.pow. Este calculo se puede ver como multiplicaciones sucesivas donde base^exp equivale a base multiplicado por base^(exp - 1). El caso base es cuando el exponente vale 0 devolviendo 1, ya que todo numero elevado a la cero es igual a 1. El caso recursivo reduce el exponente restandole 1 en cada llamada (base * potencia(base, exp - 1)). La funcion debe devolver el resultado numerico de la operacion. El codigo debe estar comentado explicando la condicion de parada, la llamada recursiva y el desapilado en memoria. Quiero probarlo con estos casos de prueba:

- potencia(5, 0)
- potencia(2, 1)
- potencia(2, 4)
- potencia(3, 3)

Ejercicio 5

Quiero implementar en Java 21 una funcion recursiva que imprima en consola un conteo desde n hasta 0. En este caso la funcion no necesariamente devuelve un valor numerico sino que es de tipo void porque debe dar una salida por consola en cada paso. El caso base es cuando n llega a 0, imprimiendo el 0 y cortando la ejecucion de la funcion. En cada paso se acerca al caso base restando n - 1. Si usaramos n + 1 el argumento se alejaria del caso base generando un bucle infinito que causaria un stack overflow. El codigo debe incluir comentarios claros de la recursion y la pila de ejecucion. Casos para probar la ejecucion:

- conteo(0)
- conteo(5)
- conteo(3)

Ejercicio 6

Quiero implementar en Java 21 una funcion recursiva para contar cuantos digitos tiene un numero entero positivo. El problema se reduce dividiendo el numero entre 10 mediante division entera (n / 10), lo que elimina su ultimo digito en cada paso. El caso base es cuando el numero es menor que 10 (n < 10) devolviendo 1, porque cualquier numero de un solo digito tiene longitud 1. En el caso recursivo cada llamada devuelve 1 + contarDigitos(n / 10) sumando una unidad por cada digito quitado. La funcion debe retornar el total de digitos encontrados. El codigo debe tener comentarios explicando el caso base, recursivo y la pila de llamadas. Se deben usar los siguientes valores de prueba:

- contarDigitos(7)
- contarDigitos(45)
- contarDigitos(12345)
- contarDigitos(0)

Ejercicio 7

Quiero implementar en Java 21 una funcion recursiva que calcule la suma de los digitos de un numero entero positivo. El problema se resuelve extrayendo el ultimo digito del numero mediante el operador modulo 10 (n % 10) y reduciendo el resto del numero mediante division entera (n / 10). El caso base es cuando el numero es menor que 10 (n < 10), devolviendo directamente el propio numero n. Los resultados se combinan sumando el digito obtenido con la llamada recursiva del resto: (n % 10) + sumarDigitos(n / 10). La funcion debe retornar la sumatoria final de todos los digitos. El codigo debe estar comentado explicando la recursion y la llamada en la pila. Pruebas para validar:

- sumarDigitos(5)
- sumarDigitos(12)
- sumarDigitos(345)
- sumarDigitos(999)

Ejercicio 8

Quiero implementar en Java 21 una funcion recursiva que invierta un String. El caso base ocurre cuando la cadena esta vacia o tiene un solo caracter (palabra.length() <= 1), donde se retorna la misma palabra porque ya esta invertida. Para reducir la cadena en cada llamada se separa el primer caracter con charAt(0) y se toma el resto con substring(1). El resultado se reconstruye al volver de la recursion concatenando la llamada recursiva del resto con el primer caracter al final: invertir(palabra.substring(1)) + palabra.charAt(0). La funcion debe retornar el String invertido completo. Debe incluir comentarios detallando el caso base, recursivo y la pila. Probarlo con estas palabras:

- "" (string vacio)
- "a"
- "hola"
- "recursivo"

Ejercicio 9

Quiero implementar en Java 21 una funcion recursiva que determine si una palabra es palindromo devolviendo true o false. El algoritmo debe comparar el primer caracter con el ultimo caracter; si son distintos la palabra no es simetrica y se devuelve false de inmediato. Si son iguales, el problema se reduce eliminando los dos extremos mediante substring(1, palabra.length() - 1) para evaluar la subcadena interna. El caso base es cuando la palabra queda con longitud 0 o 1 (palabra.length() <= 1) retornando true, ya que si todos los extremos coincidieron es palindromo. El codigo debe incluir comentarios sobre el caso base, recursivo y la ejecucion en pila. Pruebas a realizar:

- "radar"
- "reconocer"
- "a"
- “perro”

Ejercicio 10

Quiero implementar en Java 21 una funcion recursiva para buscar si un numero entero se encuentra dentro de un arreglo, sin usar ningun ciclo for ni while. La busqueda debe iniciar desde la posicion de indice 0, y para avanzar por el arreglo sin ciclos se pasa el indice incrementado en 1 (indice + 1) en cada llamada recursiva. Se contemplan dos casos base: si el elemento en la posicion actual coincide con el valor buscado (arreglo[indice] == buscado) se frena devolviendo true (o el indice hallado), y si el indice llega a ser igual a la longitud del arreglo (indice == arreglo.length) significa que llego al final sin encontrarlo y devuelve false. La funcion debe devolver el resultado de la busqueda. Debe contener comentarios claros sobre la logica, casos base y la pila de ejecucion. Probarlo con:

- [5, 8, 12], buscar 5
- [1, 4, 9, 15], buscar 9
- [2, 6, 10], buscar 7
- Arreglo vacio [], buscar 3